ADAM & MPA Consensus Security Audit Report

This report evaluates the security posture, logic, and potential runtime vulnerabilities of the implemented **ADAM (A Decentralized Approach Model)** and **MPA (Multi-Proof Algorithm)** cooperative consensus mechanisms inside the KRISTA codebase, incorporating the newly integrated multi-algorithm puzzle mining and fallback quorum scaling.

---

1. Threat Modeling: Security Against Common Attack Vectors

This section analyzes how the hybrid ADAM/MPA consensus model mitigates the most common and critical attack vectors in blockchain networks.

1.1. %51 Hashpower Attack / Cartel Monopolization

- **Traditional Vulnerability**: In standard PoW, an actor controlling 51% of the network's hashpower can double-spend, reorganize blocks, and censor transactions.
- **ADAM/MPA Mitigation**:
  - Hashpower alone is insufficient. To participate in block production, a node must be elected as one of the 11 miners or the coordinator.
  - The election pool is the Masternode network, secured by locked collateral (4,200 KRISTA).
  - To compromise a block, a cartel must control at least 7 of the 11 elected miners (threshold = 7) in a given round. This requires owning more than ~63% of the active masternode network, representing a massive financial barrier that makes the attack economically irrational.

1.2. Selfish Mining & Block Withholding

- **Traditional Vulnerability**: A miner hides mined blocks and selectively releases them to orphan honest miners' blocks, gaining an unfair share of block rewards.
- **ADAM/MPA Mitigation**:
  - Block production is collaborative. An elected miner only solves a lightweight puzzle (`vAdamSolutions`) and sends it to the coordinator.
  - A miner cannot "selfish mine" a private fork because they cannot generate the Coordinator's signature (`vAdamCoordinatorSig`) or the signatures of the other 10 miners.
  - If a miner holds their solution, the coordinator can still publish the block as long as at least 7 other elected miners submit their solutions. Selfish mining is completely neutralized.

1.3. Block Grinding / Seed Manipulation

- **Traditional Vulnerability**: In PoS chains, validators alter block content (nonces, transactions) to manipulate the next block's hash, attempting to bias the pseudo-random seed to elect themselves in future slots.
- **ADAM/MPA Mitigation**:
  - The rolling seed for the next election slot is derived from the Coordinator's deterministic VRF signature of the previous seed:

    $$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$$

- Because we use a hybrid BLS12-381 + ECDSA fallback signature mechanism (`SignBLSWithECDSAFallback`), the Coordinator has exactly one valid signature for a given seed. The BLS signature is derived deterministically from the long-term ECDSA key, and the BLS public key is signed using ECDSA to authorize it. This eliminates any degrees of freedom to grind or alter the signature value, making the next block's election seed 100% tamper-proof.

1.4. Nothing-at-Stake Attack

- **Traditional Vulnerability**: In PoS, validators can sign block headers on multiple competing forks simultaneously at zero cost, preventing fork resolution.
- **ADAM/MPA Mitigation**:
  - To validate a block on any fork, the block must contain at least 7 valid PoW puzzle solutions matching the elected miners' keys for that fork's seed.
  - Solving these puzzles requires executing real C++ hashing loops (using a 3-stage permutation of the 18 supported algorithms).
  - Because miners must spend actual physical processing power (CPU/GPU cycles) to solve the puzzles for each competing fork, the cost of staking on multiple forks is non-zero. Staking on multiple forks is computationally expensive, resolving the nothing-at-stake vulnerability.

1.5. Sybil Attacks on Node Election & Dual-Track Quota Isolation

- **Traditional Vulnerability**: An attacker spins up thousands of virtual nodes to dominate the peer list and win all leader elections.
- **ADAM/MPA Mitigation**:
  - Under the **Dual-Track Quota Election (`ElectDualTrackValidators`, active at `nAdamDualTrackHeight = UPGRADE_ADAM`)**, the 11 validator seats are partitioned into **Track 1 (Collateralized Pool — 8 Seats: 7 Active Masternodes [`4,200 KRISTA`] + 1 Coin-Lock [`250 KRISTA`] with intra-Track-1 spillover)** and **Track 2 (Open PoW Pool — 3 Seats)**. Because `ADAM_MASTERNODE_TRACK_QUOTA = 7` is enforced via compile-time `static_assert` and runtime `assert(ADAM_MASTERNODE_TRACK_QUOTA >= consensus.nAdamThreshold)` in `chainparams.cpp`, Active Masternodes alone always meet the post-bootstrap quorum threshold (`7`) even if all 4 non-Masternode seats withhold solutions; and if 5+ seats are silent simultaneously, **Standby Validator Seat Handover (`60s`)** and **Time-Relaxed Quorum Threshold (`120s`)** guarantee block recovery.
  - Even if an adversary registers hundreds of Open PoW keys, their representation is strictly capped at **3 out of 11 seats (27.3%)** whenever the Collateralized Track is populated — well below the consensus threshold (`GetAdamThreshold`: `5` during bootstrap, `7` post-bootstrap).
  - Furthermore, funded `PoW-Lock` registrations require locking at least **10 KRISTA (`POW_LOCK_REGISTRATION_AMOUNT = 10 * COIN`)** funded by UTXOs with ≥ 100 confirmations (`ADAM_POW_LOCK_MIN_INPUT_DEPTH = 100`) and are rate-limited to at most 3 `PoW-Lock` transactions per block (`ADAM_MAX_POW_LOCK_TX_PER_BLOCK = 3`). Any `PoW-Lock` output below 10 KRISTA or `Coin-Lock` output below 250 KRISTA is rejected by both mempool policy (`AcceptToMemoryPoolWorker`) and block validation (`CheckBlock`).

1.6. Signature Malleability & Relay DoS

- **Traditional Vulnerability**: An attacker mutates ECDSA signature bytes in relayed blocks to alter the block payload without changing the validity, causing cache pollution and P2P ban triggers.
- **ADAM/MPA Mitigation**:
  - The coordinator signature is excluded from the block hash (`GetHash()`), so mutating it does not affect the block's hash identity.
  - We enforce strict DER formatting checks. Any block with a mutated signature is rejected immediately at the P2P layer before entering the block index.

---

2. Technical Findings & Code Safety

2.1. Multi-Algorithm Array Bound Safety

The puzzle hashing algorithms are selected via:

$$\text{GetAdam3PermutationAlgos}(\text{hashPrevBlock}, \text{MinerPubKey}_i)$$

which deterministically maps the input to 3 distinct algorithm indices in the range $[0, 17]$.

- **Safety Check**: The `CalculateAdamPuzzleHash()` switch statement handles cases `0` to `17`, representing all 18 supported algorithms. The default branch falls back to Double-SHA256, protecting against any potential index out-of-bounds or undefined behaviors.

2.2. Masternode Fallback and Local Key Pools

- **Safety Check**: On private networks (Testnet and Regtest), if the active masternode count is low, the network falls back to a deterministic pool of keys (derived from deterministic seeds) to avoid consensus deadlocks in local sandboxes. No hardcoded key ID filtering or local wallet restrictions are applied on Mainnet, Testnet, or Regtest, ensuring a fully decentralized and trustless test environment.
- **Production Recommendation**: Ensure that on live public Mainnet, this fallback is disabled or locked. If masternodes are less than 15 on Mainnet, the chain should halt or fail to elect rather than exposing private keys, which are derivable from public seeds in the fallback logic. On Mainnet, no local key filtering or hardcoded fallback must be allowed.

2.3. CPU Denial of Service (DoS on Verification)

- **DoS Risk**: Relaying nodes must verify 11 partial puzzle signatures and one coordinator signature per block, which is CPU-intensive.
- **Mitigation**: Relaying nodes verify the block's difficulty target, transaction structure, and elected miner list *before* performing expensive signature verifications, rejecting invalid spam blocks early.

2.4. Equal-Work Fork Convergence & LLMQ Anti-Equivocation (v1.4.5.9)

- **Equal-Work Fork Resolution (`CBlockIndexWorkComparator`)**: When competing chain tips at the same height have identical `nChainWork`, nodes deterministically prefer the tip with the numerically smaller block hash (`pa->GetBlockHash() < pb->GetBlockHash()`), preventing persistent equal-work split-brain states

across network partitions.

- **LLMQ Slot Anti-Equivocation (`mapQuorumSignedSlot`)**: Quorum members record each signed `(nHeight, rotationIndex)` slot and refuse to sign a second competing `PROPOSEBLOCK` for the same slot unless the coordinator rotation index advances, ensuring at most one block per slot can obtain a 3-of-5 quorum certificate (`vQuorumSig`).
- **Strict Hot/Cold Masternode Signing Isolation**: Cold controller wallets (`masternode.conf`) are strictly isolated from consensus signing; only active Hot Nodes (`masternode=1` + `activemasternode.conf`) sign LLMQ shares, block proposals, and on-chain heartbeat pings.

---

3. Comprehensive 3-Layer Security Audit & Hardening (`v1.4.5.9` — `SEC-01` to `SEC-13`)

1. `SEC-01` **(Server File Permissions & Multi-Tenant Isolation)**: Enforced `0700` on `~/.kristatech` / `~/.kristatech-explorer` and `0600` on `kristatech.conf`, `activemasternode.conf`, `masternode.conf`, `wallet.dat`, and `.env` across all 5 nodes (`Seed 1..4` + Local).
2. `SEC-02` (`MNBROADCAST` / `MNPING` **Cache Poisoning & OOM DoS Prevention)**: Deferred insertion into `mapSeenMasternodeBroadcast` and `mapSeenMasternodePing` in `src/masternodeman.cpp` and `src/masternode.cpp` until *after* cryptographic signature (`CheckSignature`) and UTXO collateral checks succeed.
3. `SEC-03` (`ADAMSOL` & `QUORUMSIGSHARE` **Payload Size Bounds)**: Enforced strict upper bounds (`!empty() && size() <= 512`, `Misbehaving(20)`) on `ADAMSOL` (`msg.vchSolution`) before orphan caching (`mapOrphanAdamSolutions`) and on `QUORUMSIGSHARE` (`msg.vchSig`) before deserialization in `src/main.cpp`.
4. `SEC-04` (`mapFutureBlocks` **Cryptographic Pre-Verification & Queue Stuffing Protection)**: Enforced `IsFutureBlockPreVerified` (`VerifySelfContainedBLSSignedData` signature verification on `vAdamVRFProof` and `vAdamCoordinatorSig`, `vrfBlsPk == coordBlsPk`, on-chain `GetAdamMinerPool` candidate authorization, `CQuorumSignature::Verify` against `llmq::GetActiveQuorum` when `UPGRADE_MODELD` is active, unified `+300s` ceiling on both `HEADERS` and `BLOCK`, deferred `AddInventoryKnown` until `ProcessFutureBlocks`, and furthest-future eviction at `mapFutureBlocks.size() >= 64`).
5. `SEC-05` (`CQuorumSignature::Verify` **Vector & Element Bounds)**: Enforced `signatures.size() <= 32` and `sig.size() <= 512` in `CQuorumSignature::Verify` and `CQuorumSignature::AggregateBLSSignatures` (`src/llmq.cpp`), and tightened `CheckBlockHeader` (`block.vQuorumSig.size() <= 4096`) in `src/main.cpp`.
6. `SEC-06` **(Web Service Localhost Binding)**: Restricted `krista-explorer`, `krista-website`, and `kristalteknoloji.com` Nitro servers to `--host 127.0.0.1` behind Nginx TLS termination.
7. `SEC-07` **(Explorer Address Search Regex)**: Updated `Explorer/server/api/search.ts` regex from `{34}` to `/^[a-zA-Z0-9]{33,36}$/` to support 35-character KRISTA addresses (`KT...` / `kt...`).
8. `SEC-08` **(Faucet Rate-Limiting & Treasury Portal Validation)**: Added 24-hour per-IP and per-address rate-limiting (`HTTP 429`) to `KristaTech-Faucet/server/api/request.ts`, and aligned `KristaTech-Treasury-Portal` proposal validation (`rawTxHex <= 200000` hex chars, max 50 pending proposals, and 3-of-5 majority threshold matching `src/llmq.cpp`).

9. **SEC-09 (Multi-Input Developer Fund txTmp.GetHash() Normalization)**: Normalized all inputs spending from `DeveloperFundAddress()` in `CheckInputs` and `ConnectBlock` (`src/main.cpp`) by stripping the 3rd push (`qsig`) across all matching inputs before computing `txTmp.GetHash()`.
10. **SEC-10 (Disabling Legacy DECENOMY/SWMP Stateful Opcodes & CScriptContract Helpers)**: Disabled legacy `OP_MLOAD`, `OP_MSTORE`, `OP_SLOAD`, `OP_SSTORE`, `OP_PUBLISH`, `OP_RUN`, and `OP_UPDATE_STATUS` opcodes in `EvalScript` (`src/script/interpreter.cpp`) with `SCRIPT_ERR_DISABLED_OPCODE`, and hard-disabled `CScriptContract::RunContractScript`, `SaveContract`, `LoadContract`, and `UpdateContractStatus` (`src/script/script.cpp`) so they always return `false` without accessing `pScriptDB`.
11. **SEC-11 (BIP-350 Standard Bech32m Checksum Constant)**: Updated `Encoding::BECH32M` checksum constant in `src/bech32.cpp` and `KristaTech-MESCAL-Web/server/utils/mescal.ts` from `0x3fffffff` to the standard BIP-350 constant `0x2bc830a3`.
12. **SEC-12 (100% On-Chain Deterministic Consensus Elections & Sync-Independent Validation)**: Removed in-memory P2P `mnodeman` and `masternodeSync.IsSynced()` consensus dependencies across `IsValidMasternodeCollateralAndAuth`, `RegisterBlockMasternodeActivity`, `InitializeOnChainMasternodePings`, `GetAdamMinerPool`, `SelectAdamNodes` (`src/adam.cpp`), `ElectQuorumMembers` (`src/llmq.cpp`), `IsBlockPayeeValid` / `FillBlockPayee` (`src/masternode-payments.cpp`), and `CheckBlock` (`src/main.cpp`), ensuring 100% deterministic on-chain consensus state across all nodes.
13. **SEC-13 (Core Wallet TX_WITNESS_V1_TAPROOT Tracking & Coin-Selection Isolation)**: Added `TX_WITNESS_V1_TAPROOT` (`OP_1 <32-byte XOnlyPubKey>`) and general `TX_MESCAL_CONTRACT` (`keystore.HaveCScript`) evaluation to `IsMine` (`src/script/ismine.cpp`) returning `ISMINE_WATCH_ONLY` so `wallet.dat` tracks Taproot/MESCAL contract UTXOs without polluting automatic `AvailableCoins` / `SelectCoinsMinConf` selection (`ISMINE_SPENDABLE`).

---

4. Consensus Liveness, Standby Handover & Incentive Alignment Hardening (`v1.4.5.9` — `Rev 30–35`)

1. **R30-1 / R31-2 (7 + 1 + 3 Dual-Track Quotas & Runtime Assert)**: Enforced `ADAM_MASTERNODE_TRACK_QUOTA = 7`, `ADAM_COIN_LOCK_TRACK_QUOTA = 1`, and `ADAM_OPEN_POW_TRACK_QUOTA = 3` (`static_assert` in `src/adam.h` and runtime `assert(ADAM_MASTERNODE_TRACK_QUOTA >= consensus.nAdamThreshold)` in `src/chainparams.cpp`), ensuring Active Masternodes (`4,200 KRISTA`) alone satisfy the post-bootstrap quorum threshold (`7`).
2. **R31-1 / R32-2 / R32-3 / R33-3 / R34-2 (Same-Track Standby Handover at $t \geq 60$s & Time-Relaxed Threshold at $t \geq 120$s)**:
   - Added `ResolveAdamCommitteeForTime` and `ValidateAdamCommitteeForTime` (`src/adam.cpp`) allowing silent primary committee seats at `nTimeElapsed >= 60s` (`ADAM_STANDBY_ROTATION_START_SECS = 60`) to be replaced strictly by same-track standby candidates (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Open PoW -> Open PoW`) in sequential rank order verified against the block's `CCoinsViewCache* pView`.
   - Added `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` relaxing the quorum threshold by `1` every `60s` starting at t = 120s down to `ADAM_MIN_RELAXED_THRESHOLD = 5` (`> 4` non-MN seats, ensuring at least 1 Active Masternode is always required when post-bootstrap OR active

MNs ≥ 7, or `3` during bootstrap when active MNs < 7). Header-only validation (`ContextualCheckBlockHeader`) passes `0` for `nActiveMasternodes` to prevent false sync/reorg bans, while full block validation (`CheckFormsOfWork`) passes `GetActiveMasternodePoolCount(nHeight - 1, pView)`.

3. **`R33-4` / `R34-3` / `R35-2` (Equal Work Track-Specific Seat Shares, 50% Automatic Protocol Burn & Developer Treasury Split)**:
   - Under Model D (`FillBlockPayee` and `IsBlockPayeeValid` in `src/masternode-payments.cpp`), every solved committee seat (`!vAdamSolutions[i].empty()`, primary or legitimate same-track standby at $t \geq 60$s) receives its full track-specific seat share (`nPartPaymentPerSeat[i]`: `8.00%` for 1 Coin-Lock + `4.00%` per seat across 3 Open PoW = `20%`, and `~2.857%` per seat across the 7 Masternode Validator quota seats = `20%`) of the 40% Participant Validator pool (`ComputeModelDValidatorSeatPayments`).
   - Unsolved/empty seats receive **0%**, while the Coordinator retains its full 15% reward when rescuing a stalled block with standby validators. Any unclaimed empty-seat shares (`nUnclaimedPartSplit`) are split **50% to the Automatic Protocol Burn (`nExpectedBurn = nUnclaimedPartSplit / 2`)** and **50% to the Developer Treasury (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)**.

4. **`R35-1` / `R35-5` (LLMQ `qblockprop` Anti-Censorship Substitution Veto)**: Evaluated only when the LLMQ member's tip matches `msg.block.hashPrevBlock` (`pProposalView != nullptr`), at $t \geq 60$s LLMQ members veto proposals that substitute a primary seat (`vAdamMiners[i] != vExpectedMiners[i]`) whose valid solution is already cached locally (`mapAdamSolutionsCache`), while never vetoing empty primary seats (`vAdamMiners[i] == vExpectedMiners[i]`) so normal fast blocks are not delayed by in-flight P2P race conditions.
5. **`R32-1` / `R35-4` (Node-Wide Single-Block-Per-Parent Guard)**: `ActiveProposalGuard` in `src/miner.cpp` with non-destructive FIFO/age eviction (`PruneParentCoordMap`) prevents any node from concurrently or sequentially producing both a PoW and PoS block on the same parent across all block versions (`v11` and `v12`).
6. **`R36` Security Audit Hardening (`BULGU-1..13`)**:
   - **Fallback LLMQ Payee Resolution & Empty-Quorum Unclaimed Split (`BULGU-1`)**: Fallback LLMQ members without a `4,200 KRISTA` collateral UTXO are paid to their P2PKH validator destination, and if `vLlmqPayees.empty()`, `nLLMQSplitTotal` (10%) is added to `nUnclaimedPartSplit` and split **50% Automatic Protocol Burn + 50% Developer Treasury** instead of remaining with the Coordinator.
   - **Full-Committee Solver Liveness (`BULGU-2`)**: `BitcoinMiner` solver threads continue hashing across all local elected validator seats instead of breaking early at `cachedCount >= 5` on non-coordinator nodes or prior to the 30s proposal slot, eliminating premature solver starvation under `Rev 34.1` Equal-Work-Equal-Pay.
   - **Fork-Aware `pindexPrev` Ancestry & Treasury Quorum (`BULGU-3`)**: `GetAdamMinerPool` (`src/adam.cpp`) walks `pindexTip->GetAncestor(h)` from `pView->GetBestBlock()` rather than `chainActive[h]`, and `CheckInputs` (`src/main.cpp`) passes `pindexPrevSpend` and `&inputs` to `llmq::GetActiveQuorum`.
   - **Intra-Block BLS Key Consistency & VRF Pinning (`BULGU-4`)**: `CheckBlock` enforces a block-wide `mapBlockBlsKeys` consistency check across `vQuorumSig`, `vAdamSolutions`, `vAdamVRFProof`, and `vAdamCoordinatorSig`, and `RegisterBlockMasternodeActivity` pins `vAdamVRFProof`.

- **P2P DoS Protections (`BULGU-5, 6, 7, 11, 12`)**: `ADAMSOL` rejects historical/far-future heights outside `chainActive.Height() +/- 2`; `mapQuorumBlockSigs` eviction preserves seen proposed blocks; `PROPOSEBLOCK` evaluates the `R35-1` anti-substitution veto after `setSeenProposedBlocks` insertion and P2P relay; and config/key file permissions are locked to `0600` (`BULGU-10`).

---

5. Consensus Security, Decentralized PoS Co-Signing & Solver Hardening (`v1.4.5.9` — `Rev 40–41`)

1. **`R40-F1` (Qt Shutdown `powRegUnlockGuard` / `pwalletMain` Use-After-Free Fix)**: Added `DashboardWidget::shutdownMinerWorkers()` invoked inside `KRISTATECHGUI::removeAllWallets()` prior to `delete walletModel` and `Shutdown()`, joining `powRegWorkerThread` via `CancelMinerPoWRegistration()` and releasing `powRegUnlockGuard` while both `walletModel` and `pwalletMain` are alive, alongside a `ShutdownRequested() || !pwalletMain || !pcoinsTip` guard in `src/rpc/mining.cpp`.
2. **`K1` (`IsFutureBlockPreVerified` Real Cryptographic Verification on `mapFutureBlocks`)**: Hardened `SEC-04` in `src/main.cpp` by replacing non-empty byte checks with `VerifySelfContainedBLSSignedData` signature verification over `GetAdamSeed(pindexPrev)` (`vAdamVRFProof`) and `block.GetHash()` (`vAdamCoordinatorSig`), enforcing `vrfBlsPk == coordBlsPk`, verifying `coordPubKey` membership in `GetAdamMinerPool(nHeight - 1, pcoinsTip)` and `CQuorumSignature::Verify` against `llmq::GetActiveQuorum`, applying the `+300s` ceiling uniformly to both `HEADERS` and `BLOCK`, and deferring `pfrom->AddInventoryKnown` until `ProcessFutureBlocks`.
3. **`K2` (Pre-Model-D Version 12 Primary Coordinator Window Enforcement)**: Aligned `CheckBlock` (`src/main.cpp`) and `BitcoinMiner` / `CreateNewBlock` (`src/miner.cpp`) so Version 12 blocks during Pre-Model-D heights (e.g., Mainnet Blocks `2,000..2,199`) strictly enforce primary `expectedCoordinator` authority for `nBlockTimeDiff < 45s` (`ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS`) and allow PoBLS fallback rotation only once `nBlockTimeDiff >= 45s`.
4. **`K3` (`CTxMemPool::check` Sentinel `0x01:0` Assertion Bypass)**: Exempted `IsZeroCoinPoWTransaction(tx)` from `assert(pcoins->HaveCoin(txin.prevout))` inside `CTxMemPool::check` (`src/txmempool.cpp`), preventing assertion aborts on Regtest or under `-checkmempool`.
5. **`K4` (Decentralized Cooperative PoS Coordinator Co-Signing via `POSCOORDREQ` / `POSCOORDSIG` & Alternating Slot Priority)**: Added P2P messages `POSCOORDREQ` and `POSCOORDSIG` (`src/protocol.h`, `src/llmq_messages.h`, `src/adam.cpp`, `src/main.cpp`, `src/miner.cpp`) so PoS stakers whose elected Coordinator resides on a remote node obtain `vAdamVRFProof` and `vAdamCoordinatorSig` trustlessly over P2P (authenticated via `SignPoSCoordRequest` / `VerifyPoSCoordRequestAuth`, validated by the active Hot Coordinator via `TestBlockValidity`, and guarded against dual PoW+PoS equivocation via `AcquireCoordinatorProposalSlot` and a parent-alternating `1500ms` PoW/PoS slot-opening priority window `ADAM_SLOT_PRIORITY_WINDOW_MS` with `25ms` P2P signature polling).
6. **`O2`–`O5` (Single-Dispatcher Strided Solver, Deterministic `DeriveBLSFromCKey`, `0x01:0` Wallet/RPC Serialization & External Coordinator Assembly)**: Replaced duplicate per-thread `BitcoinMiner` puzzle grinding with a coordinated strided worker dispatcher (`cs_adam_solver_dispatch`), serialized `AutoRegisterMiner` and `registerminer` via `g_fMinerRegInProgress`, unified deterministic `DeriveBLSFromCKey` across all mining/registration/RPC paths, added stale `0x01:0` wallet cleanup in

`ReacceptWalletTransactions` / `ResendWalletTransactions`, and added `TryAssembleCoordinatorBlockAsync` plus `ccminer` V12 3-permutation compound puzzle support (`<a1>+<a2>+<a3>+<minerIdx>`).